

Titulación: Grado en Ingeniería Informática, Ingeniería en Sistemas de Información e InfoADE
Curso: 2024-2025. Convocatoria Ordinaria de Junio
Asignatura: Bases de Datos Avanzadas – Laboratorio

Practica 3: Seguridad, Usuarios y Transacciones.

ALUMNO 1:

Nombre y Apellidos: Gonzalo López Menor

DNI: 03210671D

ALUMNO 2:

Nombre y Apellidos: Juan Cartagena Castro

DNI: 09101498F

Fecha: 24/04/2025

Profesor Responsable: Maria Teresa Villalba

Mediante la entrega de este fichero los alumnos aseguran que cumplen con la normativa de autoría de trabajos de la Universidad de Alcalá, y declaran éste como un trabajo original y propio.

En caso de ser detectada copia, se puntuará **TODA** la asignatura como Suspenseo – Cero.

Plazos

Tarea en laboratorio: Semana 21 de abril, Semana 28 de abril, Semana 5 de mayo y semana 12 de mayo.

Entrega de práctica: **Día 16 de mayo.** Aula Virtual

Documento a entregar: Este mismo fichero con las respuestas a las cuestiones planteadas, con el código SQL utilizado en cada uno de los apartados. **Así mismo se debe de entregar el fichero de configuración postgresql.conf usado en la práctica, así como los ficheros de log de postgres (obligatorios).** Se entregará en un ZIP comprimido: **DNI 'sdelosAlumnos_PECL3.zip**

AMBOS ALUMNOS DEBEN ENTREGAR EL FICHERO EN LA PLATAFORMA.

Introducción

El contenido de esta práctica versa sobre el manejo de las transacciones en sistemas de bases de datos, así como el control de la concurrencia y la recuperación de la base de datos frente a una caída del sistema. Las transacciones se definen como una unidad lógica de procesamiento compuesta por una serie de operaciones simples que se ejecutan como una sola operación. Entre las etiquetas BEGIN y COMMIT del lenguaje SQL se insertan las operaciones simples a realizar en una transacción. La sentencia ROLLBACK sirve para deshacer todos los cambios involucrados en una transacción y devolver a la base de datos al estado consistente en el que estaba antes de procesar la transacción. También se verá el registro diario o registro histórico del sistema de la base de datos (en PostgreSQL se denomina WAL: Write Ahead Login) donde se reflejan todas las operaciones sobre la base de datos y que sirve para recuperar ésta a un estado consistente si se produjera un error lógico o de hardware. La versión de postgres a utilizar deberá ser la versión 17 preferentemente.

Actividades y Cuestiones

En esta parte la base de datos **TELPARK** deberá de ser nueva y no contener datos. Además, consta de 5 actividades:

- Conceptos generales.
- Manejo de transacciones.
- Concurrencia.
- Registro histórico.
- Backup y Recuperación

Cuestión o: Configurar el fichero de Error Reporting and Logging de PostgreSQL para que aparezcan recogidas las sentencias SQL DDL (Lenguaje de Definición de Datos) + DML (Lenguaje de Manipulación de Datos) generadas en dicho fichero. No se pide activar todas las sentencias. No activar la duración de la consulta. También se debe de configurar el log para que en el comienzo de la línea de registro de la información del log ("line prefix") aparezca el DNI de los alumnos que realizan la práctica (ambos), el nombre del host con su puerto, y la **fecha y hora** de la operación que se ha realizado.

Nos vamos a la ruta correspondiente donde esté el archivo postgres.conf

En este caso: C:\Program Files\PostgreSQL\16\data\log

Y modificamos el parámetro log_statement estableciendolo en ' mod '

Después hay que modificar el prefijo de la línea de registro line_prefix incluyendo el dni, nombre host, el puerto y la fecha y hora (%t). Esto se hace mirando lo que nos pone el archivo .conf en cuanto a instrucciones.

La duración de consulta la desactivamos con log_duration = off

Cuestión 1: ¿qué es el WAL en postgres? Si no está activo, activarlo. Determinar cuál es el directorio y el archivo donde se guarda. ¿Cuál es su tamaño? Al abrir el archivo con un editor de textos, ¿se puede deducir algo de lo que guarda el archivo?

El WAL en postgres, según el capítulo 28, es básicamente el diario del sistema. Sus siglas son WRITE--AHEAD LOGGIN. Es un mecanismo de registro de transacciones que garantizan la integridad de los datos. Antes de que cualquier cambio se escriba físicamente en las tablas, se escribe primero en los archivos WAL, permitiendo recuperación ante fallos del sistema, replicación de datos y mas seguridad de las transacciones. Comprobamos si WAL está activo(SHOW wal_level) y si no, cambiamos el parametro de wal_level en replica en el archivo .conf. El directorio es : C:\Program Files\PostgreSQL\16\data\pg_wal y su tamaño promedio(que se puede cambiar en el archivo .conf) es de 16.384 KB o 16 mb aproximadamente. Abriendolo con un editor de texto no se va a poder deducir nada; Para poder abrir el archivo y entenderlo, vamos a necesitar usar pg_waldump.

Cuestión 2: Abrir una transacción que realice una operación de inserción de un cliente y un vehículo que pertenece a ese cliente en la base de datos **TELPARK**. Cerrar la transacción con éxito. Entonces abrir el archivo de diario ¿Se encuentra reflejada la operación en el archivo del sistema? ¿Por qué?

```
BEGIN; INSERT INTO clientes (clienteid, nombre, apellido, email, telefono, provincia) VALUES  
(1, 'María', 'Pérez', 'maria.perez@email.com', 612345678, 'Madrid');
```

```
INSERT INTO vehiculos (vehiculoid, matricula, marca, modelo, color, clienteid_clientes) VALUES  
(1, '1234ABC', 'Toyota', 'Yaris', 'Azul', 1); COMMIT;
```

Se cierra con éxito. Abrimos con el editor de texto el archivo wal y no podemos identificar nada porque está encriptado básicamente. En las siguientes cuestiones veremos cómo abrir los archivos (pg_waldump)

Cuestión 3: Aplicar el comando pg_waldump.exe al último fichero de WAL que se haya generado. Obtener las estadísticas de ese fichero y comentar qué se está viendo.

Abrimos el archivo de diario con pg_waldump desde sistema operativo navegando hasta la ruta de la carpeta pg_wal y usando pg_waldump y el nombre del archivo : pg_waldump.exe

00000001000000048000000DD

Al ejecutarlo, vemos esto: rmgr: Standby len (rec/tot): 54/ 54, tx: 0, lsn: 48/DD365838,
prev 48/DD365800, desc: RUNNING_XACTS nextXid 2803 latestCompletedXid 2801
oldestRunningXid 2802; 1 xacts: 2802

Por eso, para ver las estadísticas, pondremos un -z entre pg_waldump.exe y el nombre del archivo. Esta salida nos otorga las estadísticas siguientes: Combined size, que significa la suma del tamaño de registros + FPI y porcentaje, N, que significa el número de registros de ese tipo, Record Size, que indica el tamaño en bytes que ocupan esos registros y FPI size el tamaño en bytes usados por Full Page Images si es que son usados.

El total de Combined Size es 3540425 [100%], el total de N es 1994, el total de Record Size es 169645 [4,79%] y el de FPI size es 3370780 [95,21%]

Luego tenemos en filas el XLOG(registro de control de operaciones), el Btree.. y demás tipos que conocemos.

Cuestión 4: Determinar el identificador de la transacción que realizó la operación de inserción de la cuestión 2. Aplicar el comando anterior al último fichero de WAL que se ha generado y mostrar los registros que se han creado para esa transacción. ¿Qué se puede ver? Interpretar los resultados obtenidos.

Para identificar el identificador de la transacción que realizó la operación de inserción, vamos a utilizar la siguiente consulta: `SELECT txid_current();` Con ello, nos otorga el id correspondiente(2803) y aplicamos el comando anterior sin -z al ultimo fichero WAL: `pg_waldump.exe`

00000001000000048000000DD

Podemos buscar por el id manualmente o podríamos incluir al final del comando un `| findstr "2803"`.

Al realizar el comando, obtenemos lo siguiente:

HEAP INSERT

El primer registro muestra una operación de inserción en una tabla. El registro indica que se insertaron datos en la tabla correspondiente en el bloque blk 0/12345. Esta operación tiene un tamaño de 72 bytes. Btree INSERT:

El segundo registro describe la actualización de un índice B-tree (por la insercción realizada) para reflejar la inserción de datos. La operación muestra que se insertó un registro en el índice identificado por index 16385 y que está asociado con el objeto con ID oid 29701.

COMMIT: El último registro indica que la transacción fue confirmada con éxito. La transacción con el ID 2803 se completó en el timestamp 2025-04-24 13:23:41.532918, lo que garantiza que todas las modificaciones realizadas se hicieron persistentes en la base de datos

Cuestión 5: Se va a crear un backup del clúster de postgres. Este backup será utilizado más adelante para recuperar el sistema frente a una caída del sistema. Realizar solamente el backup mediante el procedimiento descrito en el apartado 25.3 del manual (versión 17 es *"Continuous Archiving and point-in-time recovery (PITR)"*). Comentar el procedimiento realizado.

La documentación habla de que WAL produce una secuencia indefinidamente larga de registros de WAL. Dice que el sistema divide físicamente esta secuencia en archivos de segmentos WAL, normalmente en un tamaño de 16 MB etc que ya sabemos

El manual nos dice que modifiquemos los parámetros de `postgresql.conf`, `archive_mode` en ON y en `archive_command` poner la ruta al directorio de los WAL, que en este caso sería: `archive_command = 'copy "%p" "C:\\pg_wal\\%f"'`

, habilitando el archivado de los archivos WAL y definiendo el comando de windows para copiar los archivos WAL a un directorio seguro. Nos habla de otra forma de archivar utilizando un módulo de archivado personalizado como archivo, pero no lo contemplamos.

Una vez habilitado el archivado de WAL, realizamos un backup base usando `pg_basebackup`, creando una copia de la base de datos: `pg_basebackup -h localhost -D "C:\\Program`

`Files\\PostgreSQL\\16\\data\\backuptelpark" -U postgres -P --wal-method=stream`

Si quisieramos recuperarlo, el manual nos habla de que deberíamos detener el servidor, copiar todo el directorio de datos del clúster y los espacios de tabla en una ubicación temporal, eliminar todos los archivos y subdirectorios y que se restauren los archivos de la base de datos directamente en los directorios destino.

El backup: `PS C:\\Program Files\\PostgreSQL\\16\\data\\backuptelpark> pg_basebackup -h`

`localhost -D "C:\\Program Files\\PostgreSQL\\16\\data\\backuptelpark" -U`

`postgres -P --wal-method=stream`

Contraseña:

6177335/6177335 kB (100%), 1/1 tablespace

`PS C:\\Program Files\\PostgreSQL\\16\\data\\backuptelpark>`

NOTA: Tras varias cuestiones hechas, nos dimos cuenta que en los log's nos salían errores en el archivado de WAL debido a que la ruta que habíamos proporcionado era incorrecta, por lo que desde esta cuestión hasta la 11 en los logs aparecen el código de error 1, pero ya está correctamente solucionado, por lo que repetimos todas las cuestiones de nuevo por si acaso.

Cuestión 6: ¿Qué herramientas disponibles tiene PostgreSQL para controlar la actividad de la base de datos en cuanto a la concurrencia y transacciones? ¿Qué información es capaz de mostrar? ¿Dónde se guarda dicha información? ¿Cómo se puede mostrar?

PostgreSQL ofrece para controlar la actividad de la base de datos tenemos las siguientes tablas: pg_stat_activity.

Lo que muestra son los siguientes parámetros: pid, que corresponde al identificador de proceso de la conexión; username, que es el nombre de usuario de la conexión; state, que corresponde al estado de la conexión (activo, inactivo etc) y query, que es la consulta que se está ejecutando. (Además de todas las columnas que hay).

Además tenemos pg_locks que nos otorga la información sobre bloqueos y pg_stat_transactions que informa de las transacciones.

La información se guarda en esas tablas del sistema y se puede mostrar con pgAdmin.

Cuestión 7: Crear tres usuarios en la base de datos que puedan acceder a la base de datos **TELPARK** identificados como usuario1, usuario2 y usuario3 que tengan permisos de lectura/escritura a la base de datos **TELPARK**, pero que no puedan modificar su estructura. Describir el proceso seguido.

El proceso es el siguiente:

Creamos los usuarios con

```
CREATE USER usuario1 WITH PASSWORD 'usuario1';  
CREATE USER usuario2 WITH PASSWORD 'usuario2';  
CREATE USER usuario3 WITH PASSWORD 'usuario3';
```

Lo siguiente es otorgarlo los permisos de conexión a la base de datos:

```
GRANT CONNECT ON DATABASE telpark TO usuario1;
```

Damos permisos de lectura y escritura a los 3 usuarios:

```
GRANT CONNECT ON DATABASE telpark TO usuario1;
```

```
GRANT CONNECT ON DATABASE telpark TO usuario2;
```

```
GRANT CONNECT ON DATABASE telpark TO usuario3;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO usuario1;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO usuario2;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO usuario3;
```

Asegurarse de que los usuarios no puedan modificar la estructura de las tablas

```
REVOKE CREATE ON SCHEMA public FROM usuario1;
```

```
REVOKE CREATE ON SCHEMA public FROM usuario2;
```

```
REVOKE CREATE ON SCHEMA public FROM usuario3;
```

Cuestión 8: Abrir una transacción T1 en el usuario1 que realice las siguientes operaciones sobre la base de datos **TELPARK**. NO termine la transacción. Simplemente:

- Inserte un nuevo cliente de la Comunidad de Madrid con teléfono 918856931.
- Inserte un vehículo a su nombre de matrícula 1234MMG, de marca Audi, modelo A3 y color negro.

Realizar cualquier consulta SQL que muestre los datos anteriores insertados para ver que todo está correcto. Consultar la información sobre lo que se encuentra relacionado con la transacción en la actividad del sistema. ¿Qué conclusiones se pueden extraer?

Hacemos el BEGIN y las inserciones: telpark=> BEGIN;

telpark=> INSERT INTO clientes (clienteid, nombre, apellido, email, telefono, provincia)

telpark-> VALUES (2, 'Juan', 'Gómez', 'juan.gomez@email.com', 918856931, 'Madrid'); (y así con el audi)

Y comprobamos que todo está correcto: SELECT * FROM clientes WHERE clienteid = 2;

SELECT * FROM vehiculos WHERE vehiculoid = 2; (Efectivamente están bien insertados) clienteid |

nombre | apellido | email | telefono | provincia

-----+-----+-----+-----+-----+-----

2 | Juan | Gómez | juan.gomez@email.com | 918856931 | Madrid

Con el audi A3 igual.

Para consultar la información sobre lo que se encuentra relacionado con la transacción en la actividad del sistema, vamos a consultar la tabla pg_stat_activity:

Podemos observar que aparece la transacción activa: 49446 | telpark | 11400 | | 49523 | usuario1

| psql | ::1 | DESKTOP-E2PLKU9 | 54419 | 2025-05-02 18:16:43.745129+00 |

2025-05-02 18:17:27.291428+00 | 2025-05-02 18:18:17.466759+00 | 2025-05-02 18:18:17.466768+00 |

| | active | 2821 | 2821 | | SELECT * FROM pg_stat_activity; | client

backend

En donde viene la fecha y hora del empuce de la transacción y la query ejecutada, viendo que la transacción sigue activa, además no es visible el insert ni nada por otros usuarios hasta que se haga el COMMIT

Cuestión 9: Establecer una **nueva conexión** a la base de datos con el usuario2 (abrir otra sesión diferente a la abierta actualmente que pertenezca al usuario2) y realizar la misma consulta. ¿Se nota algún cambio? En caso afirmativo, ¿a qué puede ser debido el diferente funcionamiento en la base de datos para ambas consultas? ¿Qué información de actividad hay registrada en la base de datos en este momento relacionadas con las transacciones abiertas?

Abrimos una nueva conexión con el usuario2 (\c telpark usuario2) y realizamos las dos mismas

consultas: SELECT * FROM clientes WHERE telefono = '918856931';

SELECT * FROM vehiculos WHERE matricula = '1234MMG';

Se observa que no se obtiene ningún cliente que cumpla esas condiciones, debido a que todavía no se ha realizado la transacción T1 del usuario1. En PostgreSQL, las transacciones son aisladas, lo que significa que los cambios de una transacción no son accesibles para otras conexiones hasta que se complete. En cuanto a la información de actividad, vemos pg_stat_activity desde usuario2 y

observamos que : active | | 2821 | | SELECT * FROM pg_stat_activity WHERE

username = 'usuario2'; | client backend

Se ve que la transacción de usuario2 está activa y ejecutando una consulta en ese momento y además los cambios por otras transacciones no son visibles para usuario2 todavía.

Cuestión 10: ¿Se encuentran los nuevos datos físicamente en las tablas de la base de datos? Entonces, ¿de dónde se obtienen los datos de la cuestión 8 y/o de la 9?

No se encuentran los nuevos datos físicamente en las tablas de la base de datos hasta que la transacción no se haya confirmado mediante un COMMIT.

En PostgreSQL, cuando se ejecuta una transacción con BEGIN, los datos que se insertan o actualizan no son visibles para otros usuarios ni se escriben físicamente en las tablas hasta que la transacción no se confirme con COMMIT. Hasta ese momento, los datos existen en la memoria temporal de la transacción. Si se realiza un ROLLBACK, los cambios realizados durante esa transacción se deshacen y no se reflejan en la base de datos..

Cuestión 11: Finalizar con éxito la transacción T1 y realizar la consulta de la cuestión 8 y 9 sobre ambos usuarios conectados. ¿Qué es lo que se obtiene ahora? ¿Por qué?

Hacemos COMMIT, y repetimos las consultas de la 8 y 9.

Al consultar los clientes y vehículos desde cualquiera de los dos usuarios se ve que al confirmar la transacción, los cambios se han hecho visibles en la base de datos y se han confirmado, por lo que ahora tanto usuario1 como usuario2 pueden ver los cambios aplicados.

Repetimos las consultas realizadas en las cuestiones 8 y 9 para verificar si los datos se habían registrado correctamente. Al consultar las tablas de clientes y vehículos desde cualquiera de los dos usuarios conectados (usuario1 y usuario2), se observó que los cambios realizados durante la transacción ahora son visibles.

Esto sucede porque, después de ejecutar COMMIT, los cambios en las tablas (cliente y vehículo) se han hecho persistentes en la base de datos y son accesibles para todos los usuarios que estén conectados y tengan los permisos adecuados. En este caso, tanto usuario1 como usuario2 pueden ver los nuevos registros insertados, ya que los datos han sido confirmados y almacenados de forma definitiva.

Cuestión 12: Sin ninguna transacción en curso, abrir una transacción en un usuario cualquiera y realizar las siguientes operaciones:

- Insertar un nuevo cliente de la Comunidad de Cantabria.
- Insertar un nuevo vehículo con matrícula 6789HPT de ese cliente.
- Cambiar la matrícula anterior por 1234MMG.
- Insertar un nuevo cliente de la Comunidad de Madrid
- Insertar un nuevo vehículo del cliente anterior con matrícula 4567HPG
- Cerrar la transacción.

¿Cuál es el estado final de la base de datos? ¿Por qué?

El estado final de la base de datos no va a cambiar debido a que ocurre un rollback por: ERROR: llave duplicada viola restricción de unicidad «uq_vehiculos_matricula»

DETALLE: Ya existe la llave (matricula)=(1234MMG).

Ya existe un vehículo con esa matrícula entonces anula la transacción completamente.

Esto se traslada que al intentar hacer un COMMIT, ocurrirá un ROLLBACK y todo lo que anteriormente o posteriormente se intenta hacer dentro de la transacción se anula.

Cuestión 13: Repetir la cuestión 8 con otra cliente y otro vehículo que sean distintos. Realizar la misma consulta de la cuestión 9, pero ahora terminar la transacción con un ROLLBACK y repetir la consulta con los mismos dos usuarios. ¿Cuál es el resultado? ¿Por qué?

El resultado es que no se ha guardado nada dentro de la base de datos ya que se ha ejecutado al final un rollback(voluntario) para que no se quede nada guardado.

Todo esto se habla en el manual de postgresQL, cuando un usuario realiza una consulta sobre los datos dentro de una transacción que aún no se ha confirmado con un COMMIT, los cambios no son visibles para otros usuarios. Específicamente, cuando se intenta consultar los datos desde otro usuario, no verá las modificaciones hasta que la transacción esté confirmada.

Cuestión 14: Crear las siguientes tres tablas:

- Tabla1 con campo X de tipo real que sea PK.
- Tabla2 con campo Y de tipo real que sea PK.
- Tabla3 con campo Z de tipo real que sea PK.

Insertar una tupla en cada tabla con valor X=30, B=40 y C=50 respectivamente.

Creamos las tablas correspondientes e insertamos una tupla en cada tabla con los valores que nos indican:

```
CREATE TABLE Tabla1 (  
    X REAL PRIMARY KEY);
```

```
CREATE TABLE Tabla2 (  
    Y REAL PRIMARY KEY);
```

```
CREATE TABLE Tabla3 (  
    Z REAL PRIMARY KEY);
```

```
INSERT INTO Tabla1 (X) VALUES (30);
```

```
INSERT INTO Tabla2 (Y) VALUES (40);
```

```
INSERT INTO Tabla3 (Z) VALUES (50);
```

Además de que debemos otorgar los permisos necesarios de escritura/lectura

Cuestión 15: Se suministra la tabla siguiente donde se muestra la planificación que hay que seguir por tres transacciones concurrentes asociadas cada una a un usuario diferente que van a acceder en el orden marcado a las tablas anteriores. Se debe de abrir tres consolas SQL con cada uno de los usuarios (usuario1, usuario2 y usuario3) y seguir el orden de cada instrucción teniendo en cuenta que un READ es una sentencia SELECT y un WRITE es una sentencia UPDATE. Rellenar la tabla con los valores que ven cada uno de los usuarios en su ejecución de las instrucciones de cada transacción. Antes de la primera instrucción de cada transacción se debe ejecutar BEGIN;

Secuencia	T1 Usuario1	T2 Usuario2	T3 Usuario3	Valores Usuario1	Valores Usuario2	Valores Usuario3	Sentencia SQL
1		READ(Y)			40		SELECT Y FROM
2		READ(Z)			50		SELECT Z FROM
3		Y=Y/Z			0.8		SELECT Y /
4			READ(X)			30	SELECT X FROM
5			X=X+20			50	SELECT X +20 AS
6			WRITE(X)			50	UPDATE Tabla1
7		WRITE(Y)			0.8		UPDATE Tabla2
8			READ(Y)			40	SELECT Y FROM
9			Y=20*Y+X			(20*40+50)	SELECT 20 * Y +
10			WRITE(Y)			Y=850	UPDATE Tabla2
11	READ(Z)			50			SELECT Z FROM
12	Z=Z-100			-50			SELECT Z -100
13	READ(Y)			40			SELECT Y FROM
14	Y=Z/Y			1.25			SELECT(SELECT Z
15	WRITE(Z)			-50			UPDATE Tabla3
16		Z=Y+100			100.8		SELECT Y + 100
17		WRITE(Z)			100.8		UPDATE Tabla3
18	READ(X)			30			SELECT X FROM
19	X=Y*Z			-2000			SELECT(SELECT Y
20	WRITE(X)			DEADLOC			UPDATE Tabla1
21	COMMIT			ROLLBAC			COMMIT;
22		COMMIT			COMMIT;		COMMIT;
23			COMMIT			COMMIT	COMMIT;

Rellenar también la siguiente tabla con el estado del servidor y comentarios de lo que está ocurriendo en cada secuencia de ejecución de las instrucciones de cada transacción en el servidor postgres.

Estado del servidor	Comentarios
T2 lee la tabla2 T2 lee la tabla3 T2 calcula Y/Z T3 lee la tabla 1 T3 calcula $X+20$ T3 escribe en la tabla 1 T2 escribe en la tabla 2 T3 lee la tabla 2 T3 calcula $20*Y + X$ t3 escribe en la tabla 2(BLOQUEO) T1 lee la tabla 3 T1 calcula $Z-100$ t1 lee la tabla 2 T1 escribe en la tabla 3 T2 calcula $Y +100$ T2 escribe en la tabla 3(BLOQUEO) t1 lee la tabla 1 T1 calcula $Y*Z$ t1 escribe en la tabla 1 y sucede DEADLOCK T1 hace COMMIT y ocurre ROLLBACK t2 hace COMMIT exitoso T3 hace COMMIT exitoso	T2 lee el valor de Y y calcula Y/Z T3 lee la tabla 1, calcula $X+20$ y escribe en la tabla1 T2 escribe en la tabla 2 y T3 lee la tabla 2, pero no el valor actualizado que hizo T2 ya que no ha hecho COMMIT aún. T3 calcula $20*Y + X$ e intenta escribir en la tabla 2, pero se bloquea porque T2 quiere escribir en esa tabla. T1 lee la tabla 3, calcula $Z-100$, lee la tabla 2 y escribe en la tabla 3 T2 calcula $Y+100$ e intenta escribir en la tabla 3 pero se bloquea. T1 lee la tabla 1 y calcula $Y*Z$, intenta escribir en la tabla 1 y ocurre un DEADLOCK(interbloqueo) porque T1 está esperando a que T2 libere la tabla2, T2 está esperando a que T3 libere la tabla3 y T3 está esperando a que T1 libere la tabla1, por lo que mas abajo T1 intenta hacer commit pero ocurre un rollback, ya que ha sido la seleccionada para descartar(la que menos trabajo ha hecho o la más reciente). Las demás transacciones no son afectadas.

Cuestión 16: ¿Cuál es el estado final de los datos X, Y y Z en la base de datos cuando han finalizado las tres transacciones? ¿Por qué?

El estado final de los datos X,Y Y Z en la base de datos al finalizar las tres transacciones son los siguientes:

$X = 50$ | $Y = 850$ | $Z = 100.8$

Esto ocurre porque todos los cambios realizados por T1 no se han aplicado, pero en cambio T2 Y T3 sí, porque no se han descartado.

El valor de Y es 850 porque T3 hace COMMIT con el último valor que le han asignado a y. T3 a su vez no modifica Z por lo que conserva el valor que dejó T2 al hacer COMMIT. Además al hacer ROLLBACK T1, nos quedamos con el valor de X que nos dió T3.

Cuestión 17: Suponer que se produce una pérdida del cluster de datos y se procede a restaurar la instancia de la base de datos del punto 5. Realizar solamente la restauración (recovery) mediante el procedimiento descrito en el apartado 25.3.5 del manual (versión, 17) "*Continous Archiving and point-in-time recovery (PITR)*". Indicar el proceso seguido. ¿Cuál es el estado final de la base de datos? ¿Por qué?

El proceso para hacer una restauración de la base de datos del punto 5 según se comenta en el apartado 25.3.5 del manual es el siguiente:

Lo primero de todo más importante es detener el servidor.

Tras esto, es necesario que guardemos una copia del cluster(en caso de que haya espacio suficiente) o si no al menos se debe guardar el contenido del subdirectorio del clúster, ya que podría contener archivos WAL que no se archivaron antes de que el sistema se cayera.

Lo siguiente es eliminar todos los archivos y subdirectorios del directorio de datos y de los directorios raíz de los tablespaces. Esto deja el espacio listo para la restauración.

Lo que nos habla ahora es de dos opciones, instaurar una copia de seguridad completa o incremental (nosotros usamos pg_basebackup por lo que es completa), entonces copiamos el backup base directamente en el directorio de datos respetando permisos.

Ahora eliminamos todos los archivos en pg_wal provenientes de la copia de seguridad. Si no archivamos en absoluto, volvemos a crearlo con los permisos adecuados, teniendo cuidado de asegurarse de restablecerlo como un enlace simbólico si lo tenía configurado de esa manera antes.pg_wal/pg_wal/

Si tenemos archivos sin copiar que guardamos en el paso 2(copia del cluster), hay que copiarlos en archivos. Luego establecemos en postgresql.conf los valores de configuración que nos comenta (restore_command = 'copy "C:\\server\\archivedir\\%f" "%p"') y creamos un archivo recovery.signal vacío. Luego iniciamos PostgreSQL y detectará recovery signal, por lo que entrará en modo recuperación y llamará a restore_command para leer cada archivo WAL archivado y aplicarlo.

Una vez aplicado todo, eliminamos el archivo recovery.signal y pasamos a modo normal, habiendo completado todo correctamente.(Importante recalcar que hemos guardado los logs que se han producido durante todas las anteriores cuestiones para la entrega). Hacemos un select * from clientes y vemos que no hay ningún segundo cliente insertado ni ninguna tabla1 la recuperación se ha hecho

Cuestión 18: A la vista de los resultados obtenidos en las cuestiones anteriores, ¿Qué tipo de sistema de recuperación tiene implementado postgresQL? ¿Qué protocolo de gestión de la concurrencia tiene implementado? ¿Por qué? ¿Genera siempre planificaciones secuenciables? ¿Genera siempre planificaciones recuperables? ¿Tiene rollbacks en cascada? Justificar las respuestas.

PostgreSQL emplea un sistema de recuperación basado en Write-Ahead Logging (WAL). Esto significa que todas las modificaciones a la base de datos se anotan primero en el log WAL antes de aplicarse a los ficheros de datos, de modo que, tras un fallo, el servidor puede reproducir o deshacer operaciones hasta dejar la base en un estado consistente. Este sistema de recuperación completa ayuda tras caídas del servidor.

Según el capítulo 13, el protocolo de gestión de la concurrencia que tiene implementado es Multi-Version Concurrency Control (MVCC), donde cada sentencia ve una instancia de los datos ya confirmados. La ventaja principal ante bloqueo es que en MVCC los bloqueos adquiridos para consultar (leer) datos no entran en conflicto con los bloqueos adquiridos para escribir datos, por lo que la lectura nunca bloquea la escritura y la escritura nunca bloquea la lectura.

PostgreSQL busca proporcionar aislamiento serializable pero no siempre garantiza programaciones serializables en todas las situaciones, especialmente con la configuración predeterminada. En la cuestión 15 hemos podido ver que pueden ocurrir problemas de concurrencia y la posibilidad de interbloqueos incluso aunque MVCC minimice el bloqueo.

Sí, gracias a el sistema WAL de PostgreSQL que está diseñado para garantizar la recuperabilidad. Se garantiza la durabilidad de todas las transacciones confirmadas, y las transacciones incompletas pueden revertirse. El WAL garantiza que todos los cambios se registren antes de su aplicación, que ayuda mucho para restaurar la base de datos a un estado consistente tras un fallo.

Por último, como también hemos visto en teoría, PostgreSQL evita los ROLLBACK's en cascada.

Cuando una transacción se descarta, solo se deshacen sus propias operaciones, no las de otras transacciones que hayan leído los datos escritos por la transacción descartada. Esto se debe a MVCC, ya que cada transacción ve una instancia de los datos y las transacciones posteriores no dependen directamente de los cambios no confirmados.

Bibliografía

- Capítulo 9: System Information Functions and Operators, System Administration Functions.
- Capítulo 13: Concurrency Control.
- Capítulo 19: Server Configuration.
- Capítulo 25: Backup and Restore.
- Capítulo 27: Monitoring Database Activity.
- Capítulo 28: Reliability and the Write-Ahead log.